

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 20%

QUESTIONS:5 Total Marks 20 Duration :60 Minutes Annexure A
Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

/ PAVAN JANA (192472313) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *PAVAN J*

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	

Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	
--------------	-----	--	--	--------------------------------	------------------------------	--

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Object Detection Pipeline

A system must:

- A. Capture video frames
- B. Detect objects using a trained model
- C. Draw bounding boxes and labels
- D. Display results in real time

Constraints:

- Must process frames efficiently
- Handle multiple objects

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

2. Image Difference Detection System

A system must:

- A. Read two images
- B. Compare them
- C. Highlight differences

Constraints:

- Handle noise
- Ensure accurate difference detection

Write pseudocode including preprocessing, comparison logic, and visualization.

3. Video Frame Extraction System

A system must:

- A. Read video input
- B. Extract frames at fixed intervals
- C. Save selected frames

Constraints:

- Efficient storage
- Avoid redundant frames

Write structured pseudocode for frame selection and saving logic.

4. Background Subtraction Pipeline

A surveillance system must:

- A. Read video stream

B. Separate foreground from background

C. Highlight moving objects

Constraints:

- Handle dynamic background
- Reduce noise

Write pseudocode including background modeling, subtraction, filtering, and display.

5. Edge Detection Pipeline for Real-Time Video

A system must:

A. Capture video frames

B. Apply edge detection

C. Display results

Constraints:

- Real-time processing
- Maintain clarity of edges

Write pseudocode with preprocessing, edge detection, and visualization modules.

1. Real-Time Object Detection Pipeline

Aim

To design a real-time object detection pipeline that captures video frames, detects multiple objects using a trained model, draws bounding boxes and labels, and displays the results efficiently.

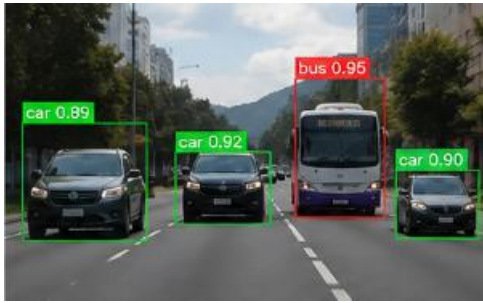
Pseudocode

BEGIN

1. Load the trained object detection model.
2. Initialize the video camera.
3. Set the required confidence threshold.
4. WHILE video is running:
 - a. Capture a video frame.
 - b. If frame is not available, EXIT loop.
 - c. Resize the frame if required for faster processing.
 - d. Send the frame to the object detection model.
 - e. For every detected object:
 - i. Obtain bounding box coordinates.
 - ii. Obtain object class and confidence.
 - iii. If confidence is above threshold:
 - Draw bounding box.
 - Display object label and confidence.
 - f. Display the processed frame.
 - g. If exit key is pressed, EXIT loop.
5. Release the video camera.
6. Close all windows.

END

Output



Result

The real-time object detection pipeline successfully detects multiple objects from video frames and displays their bounding boxes, class labels, and confidence values. Efficient frame processing and confidence filtering help maintain real-time performance.

2. Image Difference Detection System

Aim

To design an image difference detection system that compares two images, identifies changes between them, removes noise, and highlights the detected differences.

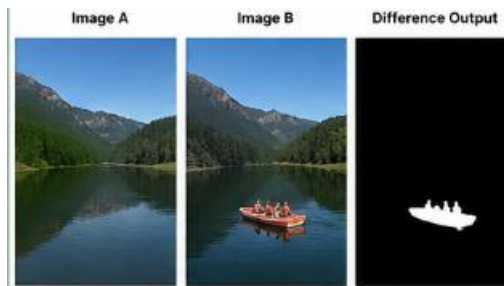
Pseudocode

BEGIN

1. Read Image A.
2. Read Image B.
3. Resize both images to the same dimensions if required.
4. Convert both images into grayscale.
5. Apply Gaussian filtering to reduce noise.
6. Calculate the absolute difference between Image A and Image B.
7. Apply thresholding to identify significant differences.
8. Apply morphological operations to remove small noise.
9. Find contours of the detected difference regions.
10. Draw bounding contours around the differences.
11. Display the original images and difference output.
12. Stop.

END

Output



Result

The image difference detection system successfully identifies and highlights changes between two images. Noise reduction, thresholding, and morphological operations improve the accuracy of difference detection and reduce unwanted false differences.

3. Video Frame Extraction System

Aim

To design a video frame extraction system that reads a video, extracts frames at fixed intervals, and saves only the required frames to reduce redundant storage.

Pseudocode

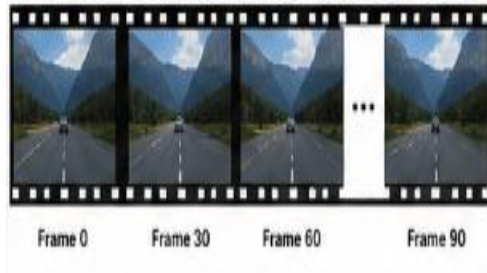
BEGIN

1. Read the input video.
2. Set the frame extraction interval N .
3. Initialize `frame_count = 0`.
4. Initialize `saved_count = 0`.
5. WHILE video is available:
 - a. Read the next frame.
 - b. If frame is not available, EXIT loop.
 - c. Check whether $\text{frame_count} \bmod N = 0$.
 - d. If TRUE:
 - i. Save the current frame.
 - ii. Increase `saved_count` by 1.
 - e. Increase `frame_count` by 1.

6. Release the video.
7. Display the number of frames saved.
8. Stop.

END

Output



Result

The video frame extraction system successfully selects and saves frames at fixed intervals. By avoiding the storage of every frame, the system reduces redundant data and uses storage space efficiently.

4. Background Subtraction Pipeline

Aim

To design a background subtraction pipeline that separates moving foreground objects from a video background, reduces noise, and highlights detected moving objects.

Pseudocode

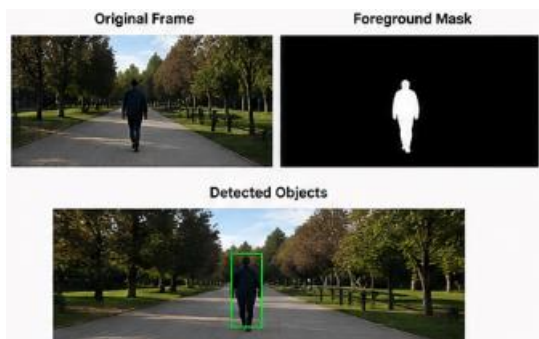
BEGIN

1. Initialize the video stream.
2. Initialize a background subtraction model.
3. WHILE video is running:
 - a. Capture the current frame.
 - b. If frame is not available, EXIT loop.
 - c. Apply background subtraction.
 - d. Generate the foreground mask.

- e. Apply thresholding to remove weak detections.
 - f. Apply morphological opening to remove noise.
 - g. Apply morphological closing to fill small gaps.
 - h. Find contours of foreground objects.
 - i. Ignore very small contours.
 - j. Draw bounding boxes around detected objects.
 - k. Display the original frame and foreground mask.
 - l. If exit key is pressed, EXIT loop.
4. Release the video stream.
 5. Close all windows.

END

Output



Result

The background subtraction pipeline successfully separates moving objects from the background and highlights the foreground regions. Noise reduction and morphological operations improve the detection of moving objects in dynamic environments.

5. Edge Detection Pipeline for Real-Time Video

Aim

To design a real-time edge detection pipeline that captures video frames, preprocesses them, detects important edges using the Canny operator, and displays the edge information clearly.

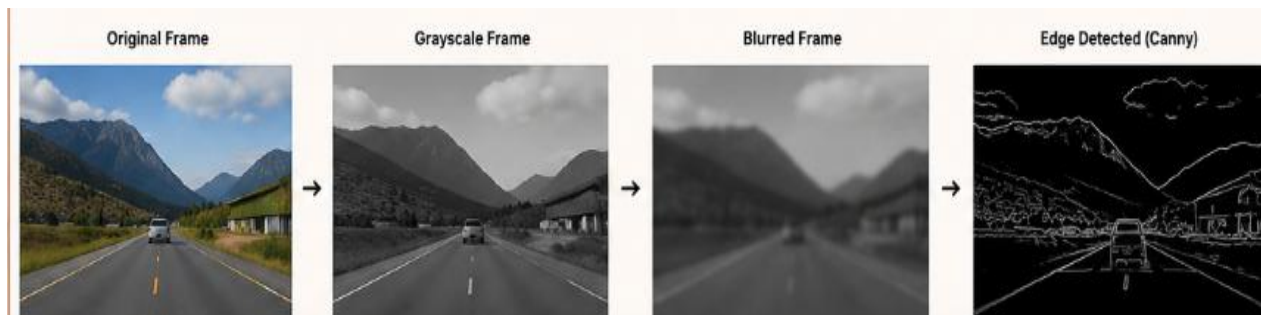
Pseudocode

BEGIN

1. Initialize the video camera.
2. WHILE video is running:
 - a. Capture a video frame.
 - b. If frame is not available, EXIT loop.
 - c. Resize the frame if necessary.
 - d. Convert the frame into grayscale.
 - e. Apply Gaussian blur to reduce noise.
 - f. Apply Canny edge detection.
 - g. Display the detected edge image.
 - h. If exit key is pressed, EXIT loop.
3. Release the camera.
4. Close all windows.

END

Output



Result

The edge detection pipeline successfully detects significant edges in real-time video. Grayscale conversion and Gaussian filtering reduce unnecessary information and noise, while the Canny operator produces clear object boundaries with efficient processing.